

《神经网络与深度学习课程》

第一次作业

发布日期：2026.03.30

提交截止时间：2026.04.12 24:00:00

作业要求：

1. 作业格式为：

（1）一个 PDF，里面按题目次序，包含回答、证明、计算过程、实验结果记录和分析等内容。

（2）一个代码文件夹，里面为实验代码。

以上的 PDF 和代码文件夹，打包压缩，发送到邮箱：weilong@fudan.edu.cn，并抄送：25112030004@m.fudan.edu.cn。邮箱标题格式为“姓名-学号-第 x 次作业”，例如“张三-20250000000-第 2 次作业”。**该邮件格式规范要严格遵守。**

2. 作业须由每个人独立完成。

3. 作业中须有一个章节“AI 的使用方式”，在此章节中，**诚实地**声明 AI 的使用方式。如果未使用 AI，则在该章节里填“无”。

4. 迟交惩罚：24 小时内扣 1 分；48 小时内扣 2 分；超过 48 小时扣 5 分。

1. 证明：在线性分类中，决策区域是凸的。即若两个样本 x_1 和 x_2 被分为某个类别 c ，则样本 $\lambda x_1 + (1 - \lambda)x_2$ 也会被分为类别 c ，其中系数 $\lambda \in (0,1)$ 。
2. 证明：对于线性可分的数据集，Logistic 回归模型的最大似然解，可通过找到一个决策边界为 $w^T x$ 且能正确分隔两类的权重向量 w （ w 可能包含偏置项），再令 w 的模长趋于无穷大得到。这里 x 为输入样本。由该结论分析，为什么通常用 w 的 L_2 范数作为 Logistic 回归的正则项。
3. 我们知道 XOR（异或）问题是线性不可分的。试设计一个前馈神经网络，来解决 XOR 问题，要求该前馈神经网络具有两个隐藏神经元和一个输出神经元，并使用 ReLU 作为激活函数。
4. 为什么在用反向传播算法进行参数学习时要采用随机参数初始化的方式，而不是直接令权重矩阵 $W = 0$ ，偏置项 $b = 0$ ？
5. 设输入特征图高 H 、宽 W 。卷积核大小 $F \times F$ ，零填充为 P （特征图四周边界各填充 P 行/列），步长 S 。计算经过这样一层卷积后，输出的高度和宽度。
6. 编程题：CIFAR-10 分类¹。请遵循 CIFAR-10 标准的训练集和测试集划分，按照以下步骤，在 CIFAR-10 上进行图片分类的编程实验。可以自行决定训练集和验证集的划分比例。
(1) 自己用 python 语言实现《神经网络与深度学习》教材中第 4 章算法 4.1 中的反向传播算法（见如下算法的截图），使其可以用于前馈神经网络（多层感知机）的分类问题，不允许使用 Pytorch 等深度学习框架中的自动微分机制，但可以调用 python 中的 numpy 等数值计算库。

¹ 下载地址：

https://www.cs.toronto.edu/~kriz/cifar.html?utm_source=chatgpt.com。

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $M_l, 1 \leq l \leq L$.

```
1 随机初始化  $\mathbf{W}, \mathbf{b}$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \dots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;  
7     反向传播计算每一层的误差  $\delta^{(l)}$ ; // 公式 (4.63)  
     // 计算每一层参数的导数  
8      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)}(\mathbf{a}^{(l-1)})^\top$ ; // 公式 (4.68)  
9      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ; // 公式 (4.69)  
     // 更新参数  
10     $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha(\delta^{(l)}(\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$ ;  
11     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;  
12  end  
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;  
    输出:  $\mathbf{W}, \mathbf{b}$ 
```

(2) 把 CIFAR-10 展平为 3072 (32x32x3) 维输入, 设计一个多层前馈神经网络 (多层感知机, 即 MLP), 用第 (1) 小题中自己实现的反向传播算法, 进行图片分类, 并对结果进行分析。

(3) 使用与第 (2) 小题相同的多层前馈神经网络, 并使用相同的初始化参数、学习率等超参数, 与第 (2) 小题的区别仅在于反向传播算法的实现, 该小题使用 Pytorch 框架中的自动微分机制。与第 (2) 小题中的结果进行对比, 以此来推断自己第 (1) 小题算法实现的正确性。

(4) 自己用代码实现图像的卷积、Pooling 操作, 并基于此编写一个不超过 10 个隐藏层的卷积神经网络, 使用 Pytorch 框架中的自动微分机制, 进行图片分类, 对分析结果, 并可视化和分析隐藏层的特征映射。